

Banking Data Model — Data Engineer's Build Spec (25-table POC)

How to read each table spec

Every table below has a one-line header with these tags:

- **Role** — what kind of table it is:
 - *Dimension* = a list of "things" (customers, branches, funds). Slowly changing.
 - *Fact* = a list of "events" (a transaction, a payment). Grows fast.
 - *Snapshot* = a periodic photo of state (balance/holding as of a date). One row per thing per period.
 - *Reference* = static lookup data.
- **Grain** — what one row means. The single most important thing to get right.
- **~Volume** — rough row count at POC scale.
- **Source** — where rows come from: *Synthetic* (we generate them), *Real API* (loaded from mfapi.in), or *Derived* (computed from other tables).
- **Layer** — the medallion layer this table's clean version lives in (bronze = raw, silver = cleaned, gold = business-ready).
- **Partition** — suggested partition column for Delta/Parquet.

Types are generic (STRING, INT, BIGINT, DECIMAL, DATE, TIMESTAMP, BOOLEAN) — map to your Fabric Lakehouse/Warehouse types.

Note on dim_date: we don't count a date dimension in the 25 — generate a standard calendar dimension in Fabric the usual way and join facts to it. It's assumed, not modelled here.

One customer (`dim_customer`) is the centre of everything. They open accounts (`dim_account`) — each account is one of three kinds: deposit, card, or loan. Money movements are recorded as **facts** (deposit transactions, card swipes, loan repayments, payments, fund purchases). Some tables are **snapshots** — periodic photos of a balance, a loan's lateness, a fund holding's value, or a customer's total wealth. Mutual Funds is special: the **funds and their prices are real data from an API**, but **who owns them is synthetic** (our generated customers buying real funds at real prices). Finally, every money movement also lands in one ledger (`gl_entry`) so the books can be reconciled.

The real-vs-synthetic line (important for the load design):

	Mutual Funds	Everything else
The "things" (funds / customers)	Funds are real (<code>dim_mf_scheme</code> , <code>dim_amc</code>)	Customers, accounts, merchants are synthetic
The prices / history	NAV history is real (<code>fact_mf_nav</code>)	—
The transactions / ownership	Synthetic (our customers buy real funds)	Synthetic

So: the mutual-fund catalog and daily prices come from [api.mfapi.in](#). Everything about *our* customers is generated. A generated customer buys a real fund at its real price on a real date.

decisions

Decision	Choice	Why it matters to you (DE)
Architecture	Medallion: bronze (raw) → silver (clean) → gold (star)	Your data-quality checks run at the bronze→silver step
The spine	One dim_account table with a account_type flag (deposit/card/loan)	You join everything back through this one table
Mutual funds	Folios link straight to the customer, not through dim_account	A fund folio isn't a bank account — don't force it into the spine
Real data	Fund catalog + full daily NAV loaded from mfapi.in	You'll build one API loader + one daily refresh job
PII	100% fake but format-valid (PAN, Aadhaar, etc.)	No real personal data anywhere — safe to share/demo
Volume	~5–6M rows, most of it in transaction + NAV facts	Facts are big, dimensions are small — partition the facts
Generation	Python scripts, seeded, generated in dependency order	The clean dataset is correct <i>by construction</i> ; faults come later

The 25 Tables

Shared spine (3 tables)

1. [dim_customer](#)

Role: Dimension · **Grain:** one row per customer · **~Volume:** 50,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none (small)

Why it exists: This is the single most important table. Every other table in all six domains links back to a customer here. One person = one row. Without this, nothing joins.

Column	Type	Plain meaning	Key
customer_id	STRING	Unique ID for the customer. The join key everything uses.	PK
customer_type	STRING	Is this a person or a business? (individual / sme)	
full_name	STRING	Customer's name (fake).	
pan	STRING	India's tax ID, format like ABCDE1234F . Fake but correctly shaped.	

Column	Type	Plain meaning	Key
aadhaar_token	STRING	A masked stand-in for the national ID — never a real one.	
date_of_birth	DATE	Birth date (or incorporation date for a business).	
kyc_status	STRING	Has their identity been verified? (verified / pending / failed)	
risk_category	STRING	Bank's simple risk label for the customer (low / medium / high).	
segment	STRING	Customer tier (retail / hni / sme).	
income_band	STRING	Rough income bracket — used for realistic behaviour.	
branch_id	STRING	The home branch they belong to.	FK → dim_branch
relationship_manager	STRING	Name of their assigned banker (folded in — we dropped the separate employee table).	
customer_since	DATE	When they joined the bank.	
status	STRING	Active / dormant / closed.	

2. dim_account

Role: Dimension (the spine) · **Grain:** one row per account · **~Volume:** 110,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none (small)

Why it exists: This is the backbone. Every bank account a customer holds — whether it's a savings account, a credit card, or a loan — gets one row here, tagged by type. The domain-specific detail lives in that domain's tables, but they all point back here. This is what lets you prove one customer's deposit, card, and loan are the same person.

Column	Type	Plain meaning	Key
account_id	STRING	Unique ID for the account.	PK
customer_id	STRING	Who owns it (the primary holder).	FK → dim_customer
account_type	STRING	The one flag that says which kind: deposit / card / loan . Decides which domain table to join to.	
product_name	STRING	The specific product (e.g. "Savings Gold", "Platinum Credit Card") — folded in from the old product table.	
branch_id	STRING	Branch the account is booked at.	FK → dim_branch

Column	Type	Plain meaning	Key
currency	STRING	Almost always INR here.	
open_date	DATE	When the account was opened.	
status	STRING	active / closed / written_off .	
close_date	DATE	When closed, if it was (nullable).	

DE note: joint accounts (multiple owners on one account) are out of scope for the POC — we keep just the primary holder as a column. Adding them later = one bridge table.

3. dim_branch

Role: Dimension / Reference · **Grain:** one row per branch · **~Volume:** 500 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: A small lookup of the bank's branches. Facts and accounts reference it so you can slice reporting by city/region.

Column	Type	Plain meaning	Key
branch_id	STRING	Unique branch ID.	PK
ifsc	STRING	India's branch routing code, format ABCD0123456 .	
branch_name	STRING	Branch name.	
city	STRING	City.	
state	STRING	State.	
region	STRING	Zone/region for roll-up reporting.	

Domain 1 — Retail Deposits (2 tables)

4. deposit_account

Role: Dimension (subtype of account) · **Grain:** one row per deposit account · **~Volume:** 70,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: Extra detail for accounts that are of type **deposit**. We folded the old separate term-deposit and overdraft tables into columns here (they're just deposits with a few extra fields), so it's one table instead of three.

Column	Type	Plain meaning	Key
account_id	STRING	Same ID as in dim_account .	PK / FK → dim_account
deposit_type	STRING	savings / checking / term / overdraft .	

Column	Type	Plain meaning	Key
<code>current_balance</code>	DECIMAL(18,2)	How much money is in it right now.	
<code>interest_rate</code>	DECIMAL(6,3)	Annual interest rate.	
<code>min_balance</code>	DECIMAL(18,2)	Minimum balance to avoid penalties.	
<code>term_months</code>	INT	For term deposits only: how long it's locked (nullable).	
<code>maturity_date</code>	DATE	For term deposits: when it unlocks (nullable).	
<code>overdraft_limit</code>	DECIMAL(18,2)	For overdraft accounts: how far they can go negative (nullable).	

5. `fact_deposit_txn`

Role: Fact · **Grain:** one row per money movement on a deposit account · **~Volume:** 500,000 · **Source:** Synthetic · **Layer:** silver → gold · **Partition:** `txn_date` (by month)

Why it exists: Every deposit, withdrawal, salary credit, ATM pull, or UPI spend on a deposit account. This is one of the highest-volume tables — treat it as a big partitioned fact.

Column	Type	Plain meaning	Key
<code>txn_id</code>	STRING	Unique ID for this transaction.	PK
<code>account_id</code>	STRING	Which account it happened on.	FK → deposit_account
<code>txn_datetime</code>	TIMESTAMP	Exact time it happened.	
<code>txn_date</code>	DATE	Date only — the partition column.	
<code>amount</code>	DECIMAL(18,2)	How much moved.	
<code>dr_cr</code>	STRING	<code>debit</code> (money out) or <code>credit</code> (money in).	
<code>running_balance</code>	DECIMAL(18,2)	Account balance right after this transaction.	
<code>channel</code>	STRING	How it was done: <code>branch</code> / <code>atm</code> / <code>netbanking</code> / <code>mobile</code> / <code>upi</code> .	
<code>narration</code>	STRING	Free-text description (e.g. "Salary July").	

DE note: `running_balance` must be internally consistent per account over time — a natural invariant you can later break for the fault demo.

Domain 2 — Cards (3 tables)

6. `card_master`

Role: Dimension · **Grain:** one row per card · **~Volume:** 55,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: One row per physical or virtual card. Credit-card billing fields (statement balance, due date) are folded in here rather than in a separate table.

Column	Type	Plain meaning	Key
<code>card_id</code>	STRING	Unique card ID.	PK
<code>account_id</code>	STRING	The card account in <code>dim_account</code> .	FK → dim_account
<code>customer_id</code>	STRING	Cardholder.	FK → dim_customer
<code>card_type</code>	STRING	<code>credit</code> / <code>debit</code> / <code>prepaid</code> .	
<code>network</code>	STRING	<code>visa</code> / <code>mastercard</code> / <code>rupay</code> .	
<code>card_token</code>	STRING	Masked card number (never store a real PAN number).	
<code>issue_date</code>	DATE	When issued.	
<code>expiry_date</code>	DATE	When it expires.	
<code>status</code>	STRING	<code>active</code> / <code>blocked</code> / <code>expired</code> .	
<code>credit_limit</code>	DECIMAL(18,2)	Spending limit (credit cards).	
<code>current_statement_balance</code>	DECIMAL(18,2)	What they currently owe (folded from the old statement table).	
<code>payment_due_date</code>	DATE	When the current bill is due.	

7. `dim_merchant`

Role: Dimension · **Grain:** one row per merchant · **~Volume:** 8,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: The shops/websites where cards get used. Lets you group spend by merchant category. The acquiring-bank detail is folded in as a column.

Column	Type	Plain meaning	Key
<code>merchant_id</code>	STRING	Unique merchant ID.	PK
<code>merchant_name</code>	STRING	Store/website name.	
<code>mcc</code>	STRING	Merchant Category Code — a standard number saying what kind of business (grocery, fuel, travel...).	
<code>category</code>	STRING	Human-readable category.	

Column	Type	Plain meaning	Key
<code>acquirer</code>	STRING	The bank that processes this merchant's card payments (folded in).	
<code>city</code>	STRING	Location.	

8. `fact_card_txn`

Role: Fact · **Grain:** one row per card transaction · **~Volume:** 300,000 · **Source:** Synthetic · **Layer:** silver → gold · **Partition:** `txn_date` (by month)

Why it exists: Every card swipe, tap, or online payment. We folded interchange fees, reward points, and dispute status **into columns here** instead of three extra tables — for a POC you rarely need them separately.

Column	Type	Plain meaning	Key
<code>card_txn_id</code>	STRING	Unique transaction ID.	PK
<code>card_id</code>	STRING	Which card.	FK → card_master
<code>merchant_id</code>	STRING	Where it was spent.	FK → dim_merchant
<code>txn_datetime</code>	TIMESTAMP	When authorised.	
<code>txn_date</code>	DATE	Date only — partition column.	
<code>amount</code>	DECIMAL(18,2)	Transaction amount.	
<code>txn_type</code>	STRING	<code>purchase</code> / <code>atm</code> / <code>refund</code> / <code>reversal</code> .	
<code>entry_mode</code>	STRING	<code>pos</code> / <code>ecom</code> / <code>contactless</code> / <code>atm</code> .	
<code>status</code>	STRING	<code>approved</code> / <code>declined</code> / <code>reversed</code> .	
<code>interchange_fee</code>	DECIMAL(18,2)	Small fee the bank earns on the transaction (folded in).	
<code>reward_points</code>	INT	Points earned on this spend (folded in).	
<code>dispute_flag</code>	BOOLEAN	Did the customer dispute this charge? (folded from the old disputes table).	

Domain 3 — Lending Lifecycle (5 tables)

9. `loan_application`

Role: Fact (event) · **Grain:** one row per loan application · **~Volume:** 35,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** `application_date` (by month)

Why it exists: Where a loan starts — an application, which may be approved or rejected. Not every application becomes a loan, which is why it's separate from `loan_account`.

Column	Type	Plain meaning	Key
<code>application_id</code>	STRING	Unique application ID.	PK
<code>customer_id</code>	STRING	Who applied.	FK → dim_customer
<code>loan_type</code>	STRING	<code>personal</code> / <code>auto</code> / <code>mortgage</code> / <code>sme</code> .	
<code>requested_amount</code>	DECIMAL(18,2)	How much they asked for.	
<code>tenure_months</code>	INT	How long they want to repay over.	
<code>application_date</code>	DATE	When they applied — partition column.	
<code>cibil_at_application</code>	INT	Their credit score at the time (300–900). This is read from <code>credit_bureau_score</code> and decides approval.	
<code>decision</code>	STRING	<code>approved</code> / <code>rejected</code> / <code>pending</code> .	
<code>approved_amount</code>	DECIMAL(18,2)	What was actually approved (nullable).	
<code>decision_date</code>	DATE	When decided.	

DE note: `cibil_at_application` being $\geq$ some threshold while `decision` = `approved` is a business rule — a good later fault ("approved despite a bad score").

10. `loan_account`

Role: Dimension (subtype of account) · **Grain:** one row per live loan · **~Volume:** 18,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: An approved application that became a real loan. Also appears in `dim_account` with type `loan`. Collections and write-off status are folded in as columns rather than separate tables.

Column	Type	Plain meaning	Key
<code>loan_id</code>	STRING	Unique loan ID.	PK
<code>application_id</code>	STRING	The application it came from.	FK → loan_application
<code>account_id</code>	STRING	Its row in the account spine.	FK → dim_account
<code>customer_id</code>	STRING	Borrower.	FK → dim_customer
<code>loan_type</code>	STRING	<code>personal</code> / <code>auto</code> / <code>mortgage</code> / <code>sme</code> .	
<code>principal</code>	DECIMAL(18,2)	Amount borrowed.	
<code>interest_rate</code>	DECIMAL(6,3)	Annual rate.	

Column	Type	Plain meaning	Key
<code>tenure_months</code>	INT	Repayment length.	
<code>emi_amount</code>	DECIMAL(18,2)	Fixed monthly payment.	
<code>disbursal_date</code>	DATE	When the money was given out.	
<code>outstanding_principal</code>	DECIMAL(18,2)	How much is still owed.	
<code>status</code>	STRING	<code>active</code> / <code>closed</code> / <code>written_off</code> .	
<code>collection_status</code>	STRING	If overdue: <code>in_collections</code> / <code>promise_to_pay</code> / null (folded from collections tables).	

11. `repayment_schedule`

Role: Fact (plan) · **Grain:** one row per installment per loan · **~Volume:** 430,000 · **Source:** Synthetic (derived from loan terms) · **Layer:** silver · **Partition:** `due_date` (by month)

Why it exists: The full month-by-month repayment plan generated when a loan starts — every future EMI, its due date, and how much of it is principal vs. interest. Big table because it's one row per month per loan.

Column	Type	Plain meaning	Key
<code>schedule_id</code>	STRING	Unique row ID.	PK
<code>loan_id</code>	STRING	Which loan.	FK → loan_account
<code>installment_no</code>	INT	1, 2, 3 ... up to tenure.	
<code>due_date</code>	DATE	When this installment is due — partition column.	
<code>emi_amount</code>	DECIMAL(18,2)	Total due this month.	
<code>principal_component</code>	DECIMAL(18,2)	Part that reduces the loan.	
<code>interest_component</code>	DECIMAL(18,2)	Part that's interest.	
<code>status</code>	STRING	<code>due</code> / <code>paid</code> / <code>overdue</code> .	

DE note: all `principal_components` for a loan should sum to `principal`. Clean invariant → good fault target.

12. `fact_loan_txn`

Role: Fact · **Grain:** one row per loan money movement · **~Volume:** 120,000 · **Source:** Synthetic · **Layer:** silver → gold · **Partition:** `txn_date` (by month)

Why it exists: Actual money events on a loan — the disbursal, each EMI paid, prepayments. Distinct from the *plan* (`repayment_schedule`) because plans and reality differ (that's the whole point of delinquency).

Column	Type	Plain meaning	Key
<code>loan_txn_id</code>	STRING	Unique ID.	PK
<code>loan_id</code>	STRING	Which loan.	FK → <code>loan_account</code>
<code>txn_date</code>	DATE	When it happened — partition column.	
<code>amount</code>	DECIMAL(18,2)	How much.	
<code>txn_type</code>	STRING	<code>disbursal</code> / <code>emi</code> / <code>prepayment</code> / <code>charge</code> .	
<code>principal_paid</code>	DECIMAL(18,2)	Portion that reduced the loan.	
<code>interest_paid</code>	DECIMAL(18,2)	Portion that was interest.	

13. `loan_delinquency`

Role: Snapshot · **Grain:** one row per loan per month · **~Volume:** 120,000 · **Source:** Synthetic (derived) · **Layer:** silver → gold · **Partition:** `snapshot_date` (by month)

Why it exists: A monthly health-check photo of every active loan: how late is it, and how bad is that. We folded the risk-provision amount in here (simplified — no actuarial columns). (*Banking aside: `risk_stage` maps to standard/underperforming/bad; you don't need to know the regulatory names to build it.*)

Column	Type	Plain meaning	Key
<code>snapshot_id</code>	STRING	Unique row ID.	PK
<code>loan_id</code>	STRING	Which loan.	FK → <code>loan_account</code>
<code>snapshot_date</code>	DATE	The month this photo is for — partition column.	
<code>days_past_due</code>	INT	How many days late the loan is (0 = on time).	
<code>dpd_bucket</code>	STRING	Simple bucket: <code>0</code> / <code>1-30</code> / <code>31-60</code> / <code>61-90</code> / <code>90+</code> .	
<code>overdue_amount</code>	DECIMAL(18,2)	How much is currently unpaid.	
<code>risk_stage</code>	INT	<code>1</code> = healthy, <code>2</code> = watch, <code>3</code> = bad. Simplified risk classification.	
<code>provision_amount</code>	DECIMAL(18,2)	Money the bank sets aside for expected loss on this loan (simplified — folded from the old ECL table).	

DE note: `risk_stage` should be consistent with `days_past_due` (a 120-day-late loan shouldn't be stage 1) — a strong cross-column fault to inject later.

Domain 4 — Payments (2 tables)

14. `dim_beneficiary`

Role: Dimension · **Grain:** one row per saved payee · **~Volume:** 90,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: The people/billers a customer has saved to pay. The "to" side of every payment.

Column	Type	Plain meaning	Key
<code>beneficiary_id</code>	STRING	Unique payee ID.	PK
<code>customer_id</code>	STRING	Who saved this payee.	FK → dim_customer
<code>beneficiary_name</code>	STRING	Payee name.	
<code>account_or_vpa</code>	STRING	Their account number or UPI ID (<code>name@bank</code>).	
<code>ifsc</code>	STRING	Payee's bank branch code (for bank transfers).	
<code>beneficiary_type</code>	STRING	<code>bank_account</code> / <code>upi</code> / <code>biller</code> / <code>international</code> .	
<code>added_date</code>	DATE	When saved.	

15. `fact_payment`

Role: Fact · **Grain:** one row per payment · **~Volume:** 200,000 · **Source:** Synthetic · **Layer:** silver → gold · **Partition:** `payment_date` (by month)

Why it exists: Every payment the customer sends. We folded the rail-specific detail (UPI IDs, SWIFT codes, biller info) and the current status **into columns** — no separate lifecycle or settlement tables for the POC.

Column	Type	Plain meaning	Key
<code>payment_id</code>	STRING	Unique payment ID.	PK
<code>from_account_id</code>	STRING	Which account paid (usually a deposit account).	FK → dim_account
<code>beneficiary_id</code>	STRING	Who got paid.	FK → dim_beneficiary
<code>rail</code>	STRING	How it was sent: <code>upi</code> / <code>neft</code> / <code>rtgs</code> / <code>imps</code> / <code>nach</code> / <code>bbps</code> / <code>swift</code> .	
<code>amount</code>	DECIMAL(18,2)	How much.	
<code>payment_datetime</code>	TIMESTAMP	When initiated.	
<code>payment_date</code>	DATE	Date only — partition column.	
<code>status</code>	STRING	<code>initiated</code> / <code>settled</code> / <code>failed</code> / <code>returned</code> (final status folded in).	
<code>reference_no</code>	STRING	The transaction reference number.	
<code>payer_vpa</code>	STRING	UPI sender ID, if rail = upi (nullable).	
<code>payee_vpa</code>	STRING	UPI receiver ID, if rail = upi (nullable).	

Column	Type	Plain meaning	Key
<code>biller_category</code>	STRING	If a bill payment: electricity/telecom/etc (nullable).	

Domain 5 — Mutual Funds (6 tables) — real funds, real prices, synthetic ownership

This is the domain that mixes real external data with our synthetic customers. Load order matters: the real fund + NAV tables must exist before any synthetic transaction can reference them.

16. `dim_amc`

Role: Reference/Dimension · **Grain:** one row per fund house · **~Volume:** ~45 · **Source:** Real API (derived from scheme metadata) · **Layer:** bronze → silver · **Partition:** none

Why it exists: The companies that run mutual funds (HDFC, SBI, Parag Parikh...). A small lookup pulled from the real fund data.

Column	Type	Plain meaning	Key
<code>amc_id</code>	STRING	Unique fund-house ID (we assign it).	PK
<code>amc_name</code>	STRING	Fund house name, straight from the API's <code>fund_house</code> .	

17. `dim_mf_scheme`

Role: Dimension · **Grain:** one row per fund scheme · **~Volume:** ~500 (curated) · **Source:** Real API · **Layer:** bronze → silver · **Partition:** none

Why it exists: The actual funds a customer can buy — real ones, loaded from `api.mfapi.in`. The `scheme_code` is the API's real ID and is our join key to prices.

Column	Type	Plain meaning	Key
<code>scheme_code</code>	STRING	The fund's real ID from the API (e.g. <code>153964</code>).	PK
<code>scheme_name</code>	STRING	Full fund name, e.g. "Parag Parikh Flexi Cap - Direct - Growth".	
<code>amc_id</code>	STRING	Which fund house runs it.	FK → <code>dim_amc</code>
<code>scheme_category</code>	STRING	Type of fund: <code>equity</code> / <code>debt</code> / <code>hybrid</code> / <code>index</code> / <code>elss</code> .	
<code>scheme_type</code>	STRING	Open/close-ended etc, from the API.	
<code>plan</code>	STRING	<code>direct</code> or <code>regular</code> .	
<code>option</code>	STRING	<code>growth</code> or <code>idcw</code> (dividend).	
<code>isin</code>	STRING	Standard security identifier from the API (nullable).	

DE note: the `/mf` list endpoint gives `scheme_code + name`; the per-scheme `/mf/{code}` call gives the richer `meta` (category, ISIN). Two-step load.

18. `fact_mf_nav`

Role: Fact (time series) · **Grain:** one row per fund per date · **~Volume:** ~1,200,000 · **Source:** Real API · **Layer:** bronze → silver · **Partition:** `nav_date` (by year or month)

Why it exists: The daily price of every fund, going back years — real market data. This is the single biggest table and the one truly external feed. Every buy/sell and every valuation reads from here.

Column	Type	Plain meaning	Key
<code>nav_id</code>	STRING	Unique row ID (<code>scheme_code + date</code>).	PK
<code>scheme_code</code>	STRING	Which fund.	FK → <code>dim_mf_scheme</code>
<code>nav_date</code>	DATE	The day this price applies to — partition column.	
<code>nav_value</code>	DECIMAL(14,5)	The fund's price per unit that day.	

DE notes: the API returns `nav` as a **string** and `date` as **DD-MM-YYYY**, newest-first — cast to decimal/date and sort ascending on load. Funds don't have a price on weekends/holidays (gaps are normal, not errors). Watch for duplicate (`scheme`, `date`) rows.

19. `mf_folio`

Role: Dimension · **Grain:** one row per customer per fund house · **~Volume:** ~40,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** none

Why it exists: A customer's "account" with a fund house — like a folder holding all their investments at that AMC. Links our synthetic customer to a real fund house. Note it links to `dim_customer` directly, **not** through `dim_account`.

Column	Type	Plain meaning	Key
<code>folio_id</code>	STRING	Unique folio ID.	PK
<code>customer_id</code>	STRING	The investor.	FK → <code>dim_customer</code>
<code>amc_id</code>	STRING	Which fund house.	FK → <code>dim_amc</code>
<code>folio_number</code>	STRING	The folio number shown to the customer.	
<code>settlement_account_id</code>	STRING	Optional link to the deposit account that funds purchases — nullable, not actively used in the POC.	FK → <code>dim_account</code>
<code>open_date</code>	DATE	When opened.	

Column	Type	Plain meaning	Key
status	STRING	active / closed.	

20. fact_mf_transaction

Role: Fact · **Grain:** one row per fund buy/sell event · **~Volume:** ~350,000 · **Source:** Synthetic (references real NAV) · **Layer:** silver → gold · **Partition:** txn_date (by month)

Why it exists: Every purchase, redemption, SIP installment, or switch. SIPs are folded in as a txn_type (a monthly SIP just produces one sip transaction per month) instead of a separate mandate table. **Each row must reference a real price:** (scheme_code, nav_date) has to exist in fact_mf_nav, and $units = amount \div nav_value$.

Column	Type	Plain meaning	Key
mf_txn_id	STRING	Unique transaction ID.	PK
folio_id	STRING	Whose folio.	FK → mf_folio
scheme_code	STRING	Which fund.	FK → dim_mf_scheme
txn_type	STRING	purchase / redemption / sip / switch_in / switch_out / dividend.	
txn_date	DATE	When the customer transacted — partition column.	
nav_date	DATE	The price date used. Must exist in fact_mf_nav.	FK → fact_mf_nav
nav_value	DECIMAL(14,5)	The price used (copied from the NAV table at that date).	
amount	DECIMAL(18,2)	Money invested or redeemed.	
units	DECIMAL(18,5)	Units bought/sold = $amount \div nav_value$.	
is_sip	BOOLEAN	Convenience flag: was this part of a recurring SIP?	

DE note: the $units = amount \div nav_value$ relationship and the "nav_date must exist in fact_mf_nav" rule are the two cleanest MF faults to inject later.

21. mf_holding_snapshot

Role: Snapshot · **Grain:** one row per folio per fund per month · **~Volume:** ~250,000 · **Source:** Derived (units × real NAV) · **Layer:** silver → gold · **Partition:** snapshot_date (by month)

Why it exists: A monthly photo of what each customer holds and what it's worth. Because fund prices move, the same units are worth different amounts each month — this table captures that so you can show gains/losses and total AUM.

Column	Type	Plain meaning	Key
<code>holding_id</code>	STRING	Unique row ID.	PK
<code>folio_id</code>	STRING	Whose folio.	FK → <code>mf_folio</code>
<code>scheme_code</code>	STRING	Which fund.	FK → <code>dim_mf_scheme</code>
<code>snapshot_date</code>	DATE	The month this photo is for — partition column.	
<code>units_held</code>	DECIMAL(18,5)	How many units they own on this date.	
<code>invested_amount</code>	DECIMAL(18,2)	Total money they put in (cost).	
<code>nav_value</code>	DECIMAL(14,5)	The fund's price on the snapshot date (from <code>fact_mf_nav</code>).	
<code>market_value</code>	DECIMAL(18,2)	What the holding is worth = <code>units_held</code> × <code>nav_value</code> .	
<code>unrealised_gain</code>	DECIMAL(18,2)	Profit/loss on paper = <code>market_value</code> – <code>invested_amount</code> .	

DE note: `market_value` must equal `units_held` × `nav_value`, and `nav_value` must match the real NAV for that scheme/date — two more clean invariants.

Domain 6 — Finance & Risk (4 tables)

22. `credit_bureau_score`

Role: Fact (time series) · **Grain:** one row per customer per score pull · **~Volume:** ~80,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** `pull_date` (by month)

Why it exists: The customer's credit score (CIBIL and similar) pulled at various times. Loan approvals read the latest one. It's a time series because scores change.

Column	Type	Plain meaning	Key
<code>score_id</code>	STRING	Unique row ID.	PK
<code>customer_id</code>	STRING	Whose score.	FK → <code>dim_customer</code>
<code>bureau</code>	STRING	Who provided it: <code>cibil</code> / <code>experian</code> / <code>equifax</code> / <code>crif</code> .	
<code>score</code>	INT	The number, 300–900. Higher = better.	
<code>band</code>	STRING	Simple label: <code>poor</code> / <code>fair</code> / <code>good</code> / <code>excellent</code> .	
<code>pull_date</code>	DATE	When it was checked — partition column.	

DE note: valid range is 300–900; anything outside is a format fault to inject later.

23. risk_alert

Role: Fact (event) · **Grain:** one row per alert · **~Volume:** ~30,000 · **Source:** Synthetic · **Layer:** silver · **Partition:** `alert_date` (by month)

Why it exists: One table for all suspicious-activity flags — we merged fraud alerts and anti-money-laundering (AML) alerts into a single table with a type column, since structurally they're the same shape.

Column	Type	Plain meaning	Key
<code>alert_id</code>	STRING	Unique alert ID.	PK
<code>customer_id</code>	STRING	Who was flagged.	FK → dim_customer
<code>alert_type</code>	STRING	<code>fraud</code> or <code>aml</code> .	
<code>related_entity_type</code>	STRING	What triggered it: <code>card_txn</code> / <code>payment</code> / <code>account</code> .	
<code>related_entity_id</code>	STRING	The ID of that transaction/payment/account.	
<code>alert_date</code>	DATE	When raised — partition column.	
<code>risk_score</code>	INT	How suspicious (0–100).	
<code>status</code>	STRING	<code>open</code> / <code>investigating</code> / <code>closed</code> / <code>false_positive</code> .	

24. customer_wealth_snapshot

Role: Snapshot · **Grain:** one row per customer per month · **~Volume:** ~150,000 · **Source:** Derived (deposits + MF holdings) · **Layer:** gold · **Partition:** `snapshot_date` (by month)

Why it exists: This is the bridge that connects Mutual Funds to Finance & Risk — the whole reason we added MF. Each month it totals up how much money the customer has with the bank (deposits + fund holdings) and segments them by wealth. It reads from `deposit_account` and `mf_holding_snapshot`.

Column	Type	Plain meaning	Key
<code>wealth_id</code>	STRING	Unique row ID.	PK
<code>customer_id</code>	STRING	The customer.	FK → dim_customer
<code>snapshot_date</code>	DATE	The month — partition column.	
<code>deposit_balance</code>	DECIMAL(18,2)	Total money in their deposit accounts.	
<code>mf_aum</code>	DECIMAL(18,2)	Total value of their fund holdings (from <code>mf_holding_snapshot</code>).	
<code>total_relationship_value</code>	DECIMAL(18,2)	Everything added up.	

Column	Type	Plain meaning	Key
<code>wealth_segment</code>	STRING	<code>mass</code> / <code>affluent</code> / <code>hni</code> , based on the total.	

DE note: `total_relationship_value` should equal `deposit_balance` + `mf_aum`, and `mf_aum` should match the sum of that customer's holdings — cross-table invariants for the fault demo.

25. `gl_entry`

Role: Fact · **Grain:** one row per ledger posting · **~Volume:** ~1,500,000 · **Source:** Derived (from every money movement) · **Layer:** silver → gold · **Partition:** `posting_date` (by month)

Why it exists: The bank's single ledger. Every money movement across all six domains — deposit txns, card txns, loan txns, payments, fund transactions — also writes a row here, so the books can be reconciled. The chart-of-accounts is folded in as a `gl_code` + `gl_category` column rather than a separate reference table.

Column	Type	Plain meaning	Key
<code>gl_entry_id</code>	STRING	Unique posting ID.	PK
<code>gl_code</code>	STRING	Which ledger account this hits (folded chart-of-accounts).	
<code>gl_category</code>	STRING	Grouping: <code>asset</code> / <code>liability</code> / <code>income</code> / <code>expense</code> .	
<code>source_domain</code>	STRING	Which domain it came from: <code>deposit</code> / <code>card</code> / <code>loan</code> / <code>payment</code> / <code>mf</code> .	
<code>source_txn_id</code>	STRING	The original transaction's ID (links back to the domain fact).	
<code>amount</code>	DECIMAL(18,2)	Posting amount.	
<code>dr_cr</code>	STRING	<code>debit</code> or <code>credit</code> .	
<code>posting_date</code>	DATE	When posted — partition column.	

DE note: for any source transaction, debits and credits should balance. "Debits ≠ credits" is the classic reconciliation fault.

Building it

Medallion layers — where each table lives

- **Bronze (raw, append-only):** raw landing of everything. The real MF API payloads land here first (raw scheme JSON, raw NAV arrays) exactly as received. Synthetic source extracts land here too. No cleaning.
- **Silver (clean, conformed):** typed, deduplicated, keys enforced. NAV strings → decimals, DD-MM-YYYY → dates. This is where your **data-quality checks run** (the bronze→silver step). Almost every table's trustworthy version lives here.
- **Gold (business-ready star schema):** the facts + dimensions the dashboards read, plus the derived snapshots (`mf_holding_snapshot`, `customer_wealth_snapshot`) and the reconciliation view over

gl_entry.

Load / generation order (the dependency graph)

You must build tables in this order — each step needs the previous ones to exist:

1. Reference + spine
dim_branch → dim_customer → dim_account
2. Real MF data (independent, can run in parallel with step 1)
dim_amc → dim_mf_scheme → fact_mf_nav [load from API]
3. Deposit + card + payment detail
deposit_account, card_master, dim_merchant, dim_beneficiary
4. Domain facts
fact_deposit_txn, fact_card_txn, fact_payment
5. Lending chain
loan_application → loan_account → repayment_schedule
→ fact_loan_txn → loan_delinquency
6. MF ownership (needs real MF data from step 2)
mf_folio → fact_mf_transaction → mf_holding_snapshot
7. Risk + derived
credit_bureau_score, risk_alert
customer_wealth_snapshot (needs deposits + MF holdings)
gl_entry (needs all money-movement facts)

Why order matters: you can't create a fund transaction (step 6) before the fund's prices exist (step 2); you can't build a delinquency snapshot (step 5) before the loan exists; you can't total someone's wealth (step 7) before their deposits and holdings exist. Generating in this order makes the clean dataset correct by construction.

Volume plan

Table	~Rows	Big?
dim_customer	50,000	
dim_account	110,000	
dim_branch	500	
deposit_account	70,000	
fact_deposit_txn	500,000	✓ partition
card_master	55,000	
dim_merchant	8,000	
fact_card_txn	300,000	✓ partition
loan_application	35,000	
loan_account	18,000	

Table	~Rows	Big?
repayment_schedule	430,000	✓ partition
fact_loan_txn	120,000	
loan_delinquency	120,000	
dim_beneficiary	90,000	
fact_payment	200,000	✓ partition
dim_amc / dim_mf_scheme	45 / 500	real
fact_mf_nav	1,200,000	✓ real, biggest
mf_folio	40,000	
fact_mf_transaction	350,000	✓ partition
mf_holding_snapshot	250,000	✓ partition
credit_bureau_score	80,000	
risk_alert	30,000	
customer_wealth_snapshot	150,000	✓ partition
gl_entry	1,500,000	✓ biggest derived
Total	~5.7M	

Small dimensions, big facts — partition the facts by their date column, leave dimensions unpartitioned.

Fabric build sequence (short)

1. Provision workspace + capacity (F-SKU or trial).
2. Create a Lakehouse; land bronze (generated Parquet + raw MF API JSON).
3. Build the **MF API loader** (one-time full history) and a **daily NAV refresh** pipeline.
4. Data Factory master pipeline: land bronze → run silver notebooks (clean/type/dedupe/validate keys) → publish gold.
5. Warehouse for the gold star schema + semantic model.
6. Power BI dashboards: customer 360, deposits, cards, lending & risk, payments, **MF portfolio & AUM**.

What we can break later (fault-injection preview)

The model was shaped so these faults are natural to inject at the bronze→silver step:

- **Bad references:** MF transaction pointing at a `nav_date` we never loaded; holding for a scheme not in `dim_mf_scheme`; payment from a closed account.
- **Broken math:** `units ≠ amount ÷ nav_value`; `market_value ≠ units × nav_value`; schedule principal not summing to loan principal; ledger debits ≠ credits.
- **Bad values:** credit score outside 300–900; NAV negative or non-numeric; wrong date format from the feed.
- **Duplicates / gaps:** duplicate NAV for a scheme/date; missing NAV on a trading day.

- **Inconsistent state:** `risk_stage = 1` on a 120-day-late loan; wealth total not matching its parts.
 - **Feed problems (real data):** the daily NAV feed genuinely has gaps, holidays and late updates — the most credible place to demo self-healing.
-

Appendix — noted for the fault-handling phase (not built now)

When we scope the self-healing phase, add two operational tables (kept out of the model for now):

- `dq_quarantine` — rows that failed a quality check, parked instead of dropped.
- `dq_correction_log` — an audit trail of every automatic fix (what was wrong, what it was changed to, when), so corrections are reversible and explainable.